

Università di Padova
Corso di
Ingegneria del Software
A.A. 2026/2027



Gruppo 18
Lord of the Strings
thelordofthestrings03@gmail.com

Valutazione dei capitolati proposti

Storico modifiche

Versione	Data	Descrizione	Autore
1.0.0	2026-10-29	Approvazione finale.	Tutto il gruppo
1.0.0	2026-10-29	Verifica del documento.	Giacomo De Lucchi
0.1.5	2026-10-24	Valutazione capitolati C3 e C4.	Yi Ming Ye
0.1.4	2026-10-24	Valutazione capitolato C9.	Giovanni Biba
0.1.3	2026-10-23	Valutazione capitolato C5.	Stefano Sironi
0.1.2	2026-10-23	Valutazione capitolati C7 e C8.	Costanza Gyulay
0.1.1	2026-10-22	Valutazione capitolati C0 e C1.	Giacomo De Lucchi
0.1.0	2026-10-21	Frontespizio, indice, struttura del documento, valutazione capitolati C2 e C6.	Michele Moro

Indice:

1 Valutazione del capitolato scelto.....2
1.1 Capitolato C5: RareCare.....2
2 Valutazione dei capitolati rimanenti.....3
2.1 Capitolato C0: AIRA.....3
2.2 Capitolato C1: AURA.....4
2.3 Capitolato C2: Concierge.....5
2.4 Capitolato C3: Fossdot.....6
2.5 Capitolato C4: Gestione veicoli e servizi post-vendita.....6
2.6 Capitolato C6: Dashboard FinOps.....7
2.7 Capitolato C7: Smart Dashboard.....8
2.8 Capitolato C8: Sonar.....9
2.9 Capitolato C9: SpecCheck.....10

1 Valutazione del capitolato scelto

1.1 Capitolato C5: RareCare

Descrizione:
Il progetto consiste nello sviluppo di una piattaforma digitale dotata di un assistente virtuale basata su intelligenza artificiale per supportare la gestione del percorso di cura di malattie rare e complesse, nello specifico della sindrome di Sindrome di Sanfilippo (Mucopolisaccaridosi di tipo III).
Il software deve avere le seguenti funzionalità:

- assistente virtuale AI: interpreta protocolli sanitari e pratiche amministrative guidando le famiglie verso i servizi territoriali competenti;
- gestisce accessi profilati per connettere tutti gli attori della rete di cura;
- gestione scadenze e promemoria: tracciamento di visite, esami, rinnovi di benefici di legge;
- archivio cifrato per conservare referti, piani terapeutici e scolastici, documenti amministrativi.

Tecnologie consigliate:
Sono state indicate molte possibili tecnologie per l'implementazione del progetto, in seguito le principali:

- frontend: Flutter o React Native;
- accesso e sicurezza: Amazon Cognito per gestione identità e autenticazione, Amazon Verified Permissions;
- AI: Amazon Bedrock e relativi servizi per accesso a modelli LLM, gestione filtri e pipeline RAG;
- dati: Amazon Aurora per gestione dati strutturati, Vector Store per ricerca semantica nel RAG, Amazon S3 per la gestione dei fascicoli riservati.

Motivazione della scelta:
È stato scelto questo progetto dalla maggior parte del gruppo per le seguenti motivazioni:

- progetto con un grande valore sociale in grado di aiutare le famiglie con membri affetti da malattie rare;
- possibilità di lavorare con strumenti AI;
- è stata percepita una grande disponibilità da parte dell'azienda all'affiancamento e al supporto per lo sviluppo del progetto;
- possibilità di lavorare con servizi AWS, leader del mercato mondiale di servizi cloud.

2 Valutazione dei capitolati rimanenti

2.1 Capitolato C0: AIRA

Descrizione:
Progetto AIRA ha come obiettivo lo sviluppo di una piattaforma che utilizza l'intelligenza artificiale per estrarre ed interpretare informazioni da file e documenti provenienti da vari canali informativi utilizzati tra clienti e azienda.
Questi canali possono essere testuali, come mail con allegati PDF, messaggi provenienti da una chat WhatsApp, oppure audiovisivi, ad esempio fotografie di documenti oppure note vocali.
Una volta estratte le informazioni, il software le dovrà memorizzarle in maniera strutturata e successivamente inoltrarle automaticamente alle piattaforme gestionali aziendali che devono farne uso.
La piattaforma deve inoltre avere un'interfaccia web amministrativa per configurare i canali, gestire le regole, consultare i log e mantenere i dati. Sono richiesti anche test unitari e di integrazione, documentazione e codice versionato. Dashboard analitica e risposte automatiche al mittente sono invece funzionalità opzionali.

Tecnologie consigliate:
- Angular e TypeScript per la realizzazione dell' interfaccia amministrativa;
- Spring Boot e Java per lo sviluppo del core del software;
- Database SQL come SQLite, MariaDB e PostgreSQL per la persistenza dei dati;
- LangChain4j come framework Java per integrare gli LLM richiesti nel prodotto software;
- Whisper e Google Speech per il riconoscimento del parlato;
- Tesseract e EasyOcr estrarre testo da immagini e PDF scansionati;
- Github o Bitbucket come sistemi di versionamento e per la consegna del prodotto;
- API REST per collegare il sistema ai software aziendali già esistenti, come ERP, CRM, portali B2B e piattaforme di ticketing.

Inoltre viene suggerito di adottare una metodologia Agile per favorire la collaborazione del gruppo di lavoro.

Pro:
- Progetto che se realizzato darebbe un enorme contributo amministrativo alle aziende, riducendo di molto il tempo impiegato a raccogliere e riordinare le informazioni dai vari canali informativi.
- Gestione di canali differenti: il sistema può ricevere informazioni da e-mail, messaggi, audio, immagini e documenti.
- In futuro avrebbe la possibilità di estendere il sistema aggiungendo canali o tipologie di output differenti.
- L'estrazione automatica e la validazione dei dati possono limitare errori di trascrizione, campi dimenticati e informazioni inserite nel sistema sbagliato.
- Integrazione con i sistemi aziendali già presenti

Attraverso le API REST, AIRA può trasferire le informazioni a ERP, CRM, portali B2B e sistemi di ticketing, evitando di sostituire necessariamente gli strumenti già utilizzati dall'azienda.

Contro:

- La documentazione fornita non è molta, e si limita alle sole slide utilizzate in fase di presentazione.
- Viene chiesto di presentare documentazione relativa ai test effettuati, ma non si specifica la tipologia di questi, né la coverage minima.
- Non è presente una definizione di MVP, né di PoC.
- Così come presentato i canali informativi in input sembrano troppi e troppo diversificati per una consegna nei tempi e costi del progetto.
- Non è chiaro quali canali in input e sistemi gestionali aziendali saranno effettivamente disponibili per lo sviluppo e per la dimostrazione finale: questa informazione incide sensibilmente sulla fattibilità e sulla ripartizione del lavoro del gruppo.

2.2 Capitolato C1: AURA

Descrizione:
Il progetto AURA (AI for Understanding, Reasoning and Automation) mira a integrare servizi di intelligenza artificiale generativa commerciali con i sistemi domotici Vimar, permettendo agli utenti di controllare e interrogare i dispositivi della casa tramite linguaggio naturale, dialogando tramite la chat IA.
Il sistema dovrà utilizzare un server MCP per collegare l'intelligenza artificiale alle API REST di KNX IoT, consentendo di leggere e modificare lo stato dei dispositivi domotici Vimar.
La soluzione dovrà essere containerizzata, predisposta al deployment cloud e accompagnata da documentazione, test e istruzioni di installazione.
Vimar fornisce un kit fisico portatile di impianto domotico, composto da sensori, comandi e attuatori, utile per eseguire test durante lo sviluppo.

Pro:

- Ottima documentazione della presentazione capitolato: l'azienda ha fornito un documento di presentazione capitolato, esteso e che tratta molti punti.
- Supporto aziendale strutturato: Vimar garantisce un kit fisico di prova, referenti dedicati e incontri periodici di SAL (Stato Avanzamento Lavori), riducendo il rischio di bloccarsi su problemi tecnici.
- Specifica chiara e completa delle principali tecnologie da adottare
- Fornitura di un kit fisico portatile di impianto domotico per realizzare test in fase di sviluppo, utilizzando un impianto concreto.
- Vimar si è dimostrata molto disponibile e attenta ai dettagli

Contro:

- Molta competizione con gli molti altri gruppi per la fase di candidatura, rischio concreto di non aggiudicazione

2.3 Capitolato C2: Concierge

Descrizione:
Il progetto si propone di integrare un assistente AI all'interno di una piattaforma HR per semplificare l'accesso alle informazioni e ai servizi aziendali attraverso il linguaggio naturale. Per mantenere il percorso ad un livello di complessità affrontabile, il Minimum Viable Product si concentra esclusivamente sull'area "ferie e permessi". L'assistente integra quattro capacità principali: consultare i regolamenti aziendali, fornire istruzioni sul prodotto NEXUM, leggere i dati personali aggiornati dell'utente e guidarlo nella registrazione delle richieste.

Tecnologie consigliate:
- Nexum, è una piattaforma hr già esistente.
- Progressive Web App per l’interfaccia utente.
- Amazon Bedrock.
- Claude Code o Codex.

Pro:
- Il progetto ha un ambito limitato: la limitazione del MVP all'ambito di ferie e permessi riduce il rischio di ampliamento dei requisiti e garantisce una soluzione verificabile.
- Prevede un modo per prevenire le allucinazioni LLM: l’assistente AI da integrare in Nexum non può inventare nulla ma deve interrogare servizi applicativi attendibili, evitando di rispondere se le fonti autorevoli sono insufficienti o contrastanti.
- È previsto il controllo delle autorizzazioni aziendali per l'accesso ai dati, il tracciamento delle fonti citate e l'obbligo di mostrare un riepilogo con richiesta di conferma esplicita prima di eseguire ogni operazione.

Contro:
- Il completamento delle operazioni e l'affidabilità dei conteggi orari si basano totalmente sulle API del servizio Nexum, necessitando di forte supporto da parte dell'azienda per la fase di sviluppo.
- Eggon prevede 480 ore preventivate a persona, tempo che supera il monte ore previsto dal progetto didattico.

2.4 Capitolato C3: Fossdot

Descrizione:
Il progetto richiede la creazione di un ecosistema open source per gli smart speaker, articolandosi nel modo seguente:

1. selezionare hardware: individuare la piattaforma hardware di riferimento tenendo conto anche del fattore forma in modo che i componenti possano essere racchiusi in un contenitore adatto al prodotto finale;
2. sviluppo firmware per rilevamento audio, wake word, streaming audio verso il server e risposta;
3. server di bridging: ponte verso provider AI con livello astrazione per rendere provider AI configurabile e sostituibile;
4. interfaccia registrazione dispositivi: applicazione multi-dispositivo per associare Fossdot ad un account per gestire le impostazioni.

Tecnologie consigliate:
Piena libertà implementativa, sviluppo firmware preferibilmente in Rust.

Pro:

- progetto interessante;
- fornitura di sessioni di approfondimento tecnico da parte del committente.

Contro:

- progetto troppo complesso e vasto, richiede competenze in ambiti eterogenei, si rischia di superare l'impegno orario previsto;
- difficoltà nella scelta dell'hardware oltre a dover tener conto anche della forma;
- grande responsabilità nella gestione dei dati sensibili, ai sensi del GDPR.

2.5 Capitolato C4: Gestione veicoli e servizi post-vendita

Descrizione:
Creazione di una piattaforma software distribuita per la gestione di informazione sui veicoli, campagne di richiamo, manutenzioni e servizi vari. L'obiettivo è migliorare la collaborazione tra casa automobilistica, concessionaria e cliente finale.

Tecnologie consigliate: NodeJs, NestJs, TypeScript, Redis, Go, Nats, Kubernets, MongoDB, Angular.

Pro:

- progetto con struttura semplice.

Contro:

- progetto poco stimolante per il gruppo;
- presentazione superficiale e poco approfondita, documentazione approfondita del capitolato non reperibile al momento della stesura di questo documento.

2.6 Capitolato C6: Dashboard FinOps

Descrizione:
Il progetto si propone di estendere il proxy open source LiteLLM integrando due nuovi moduli per ottimizzare l'uso aziendale dell'Intelligenza Artificiale: un sistema di caching semantico e una dashboard FinOps. Gli obiettivi sono due: ridurre il consumo dei token riconoscendo le domande con lo stesso significato ma formulate diversamente, e fornire all'azienda una visione strutturata e aggregata sui costi dei vari progetti e team.

Tecnologie consigliate:

- LiteLLM,
- Python e FastAPI o Flask,
- Docker e Docker Compose,
- Redis,
- Qdrant o pgvector,
- React e Chart.js.

Pro:

- di particolare interesse è il sistema di riduzione del consumo di token: il modello non viene interrogato più volte per domande equivalenti, evitando di pagare ripetutamente per le stesse risposte. La cache semantica sfrutta gli embedding per rilevare le richieste simili e restituire il dato già salvato.
- Il progetto è basato sull'uso di LiteLLM, un orchestratore già in produzione che fa da interfaccia unica compatibile con lo standard OpenAI.

Contro:

- complessità della cache: si richiede la corretta rilevazione della soglia di similarità semantica per ottenere un corretto bilanciamento tra efficienza e correttezza delle risposte. Richiede inoltre una gestione efficiente dei tempi di vita dei dati salvati (TTL) e del loro versionamento.
- L'ecosistema risulta molto complesso ed è richiesta l'integrazione di diversi elementi: modelli di embedding, proxy scritti in Python, strumenti per la similarity search come Qdrant o pgvector e database di supporto come Redis.

2.7 Capitolato C7: Smart Dashboard

Descrizione:
Il progetto prevede lo sviluppo di Smart Dashboard, una piattaforma di Business Intelligence che consente di analizzare i dati aziendali attraverso dashboard interattive e personalizzabili. Il sistema integra funzionalità di intelligenza artificiale per generare visualizzazioni a partire da richieste in linguaggio naturale e individuare anomalie e tendenze nei dati.

Tecnologie consigliate:
- data warehouse: DuckDB (alternative: ClickHouse, MonetDB, Apache DataFusion);
- database relazionale: PostgreSQL, MySQL, MariaDB;
- ETL e trasformazione dati: Python, Polars;
- connessione al gestionale: ODBC, middleware con scambio JSON;
- backend: framework a scelta, API REST;
- frontend: React, Angular, Vue;
- visualizzazione dati: librerie di charting e librerie per layout a griglia con supporto drag & drop;
- intelligenza artificiale: API LLM come GPT di OpenAI o Claude di Anthropic.

Pro:
- possibilità di lavorare su aspetti diversi dello sviluppo software, dal database al backend e al frontend, fino all'integrazione dell'AI;
- libertà nella scelta delle tecnologie e nella progettazione dell'architettura;
- possibilità di dividere il lavoro in fasi e assegnare compiti diversi ai membri del gruppo, grazie alla suddivisione del sistema in componenti separati;
- disponibilità di un possibile caso di studio reale, utile per provare la piattaforma su dati concreti.

Contro:
- progetto molto ampio, che comprende sia la realizzazione di una piattaforma BI completa sia l'integrazione di diverse funzionalità AI;
- maggiore complessità nell'integrazione dell'AI, che non deve solo generare risposte, ma anche interpretare correttamente i dati aziendali, creare visualizzazioni coerenti con le richieste e individuare anomalie e tendenze senza trarre conclusioni errate.

2.8 Capitolato C8: Sonar

Descrizione:
Il progetto Sonar prevede lo sviluppo di una piattaforma che analizza automaticamente uno store Shopify, ricostruendone la struttura e le dipendenze tra app, automazioni, dati e componenti del tema. Il sistema genera una documentazione aggiornata dello store e segnala configurazioni inutilizzate, codice non più necessario e possibili problemi tecnici.

Tecnologie consigliate:
- framework e hosting: Next.js (App Router) e Vercel;
- orchestrazione dei processi: Vercel Workflows e Vercel Cron Jobs;
- database: Supabase;
- linguaggi e API: TypeScript, Shopify Admin API (GraphQL);
- frontend: React, Tailwind CSS, shadcn/ui;
- visualizzazione dei grafi: React Flow o Cytoscape.js;
- test: Vitest, Playwright, MSW;
- CI/CD e monitoraggio: GitHub Actions, Vercel Observability, OpenTelemetry.

Pro:
- libertà nella progettazione dell'architettura software, con la possibilità di definire autonomamente la struttura del sistema e le soluzioni architetturali più appropriate;
- opportunità di apprendimento e approfondimento tecnico, in particolare per quanto riguarda l'integrazione tramite API, la normalizzazione di dati eterogenei e l'inferenza a partire da evidenze parziali.

Contro:
- necessità di approfondire una tecnologia molto specifica (Shopify), che richiede uno studio dedicato e che potrebbe risultare poco stimolante per i membri del gruppo che non sono particolarmente interessati a questo ambito;
- elevato carico di lavoro complessivo, che comprende anche attività preliminari impegnative, come la realizzazione di uno store articolato e sufficientemente complesso da consentire lo sviluppo e la verifica delle funzionalità richieste;
- elevata complessità progettuale e implementativa, legata alla costruzione del grafo, allo sviluppo del motore di regole per la generazione delle segnalazioni, all'integrazione e alla triangolazione di fonti informative diverse e alla definizione di livelli di confidenza per le evidenze raccolte;
- numero elevato di requisiti da analizzare e implementare, che potrebbe rendere più difficile la pianificazione delle attività, la gestione delle priorità e il rispetto delle tempistiche previste.

2.9 Capitolato C9: SpecCheck

Descrizione:
Il progetto SpecCheck prevede lo sviluppo di un insieme coordinato di estensioni (*skill*, *tool*, *comandi* e *plugin*) per l'agente di gestione OpenCode. L'obiettivo principale è verificare in modo automatico la tracciabilità e la coerenza end-to-end lungo l'intera durata dello sviluppo software: dai diagrammi dei casi d'uso e la relativa documentazione, passando per le specifiche dei requisiti, fino al codice sorgente finale. Il sistema valuta un insieme di metriche e le espone tramite una dashboard al programmatore.

Tecnologie consigliate:

- agente/Harness di riferimento: OpenCode
- estensioni del sistema: OpenCode Skills, Tools, Commands e plugin/applicativo esterno;
- formati di input (Casi d'uso e Requisiti): Markdown, Mermaid, XMI (XML per UML), immagini (PNG/JPEG) *(opzionale: documenti PDF)*
- modelli LLM: modelli proprietari via API e modelli *open-weight* locali (Gemma 4 26B, Qwen 3.8 27B);
- linguaggi e Tool di supporto: script di installazione (Bash/PowerShell), linguaggi a scelta per il plugin/eseguibile esterno (es. Go, Rust, Python, Node.js);
- testing e Automazione: Test automatici per la verifica delle regressioni durante le iterazioni (PoC e MVP).

Pro:

- visibilità ed eco open-source: È prevista la possibilità di pubblicare il progetto su GitHub in modalità open-source, aumentando la visibilità e l'effettivo utilizzo del lavoro svolto;
- opportunità di apprendimento avanzato: Permette di approfondire tecnologie e architetture all'avanguardia legate agli agenti AI, all'integrazione di *harness* (come OpenCode) e alla gestione formale del contesto di progetto, spostando l'attenzione dalla semplice generazione di codice alla verifica e al controllo di qualità.

Contro:

- assenza di supporto formativo e affiancamento fisico: L'azienda fornisce le risorse tecniche e gli accessi (modelli LLM, API, budget di calcolo), ma non offre un affiancamento o supporto funzionale all'apprendimento; il gruppo deve informarsi in totale autonomia sul funzionamento di OpenCode, dei framework e dei modelli.
- Ambito del progetto poco stimolante.